

DESIGN SPECIFICATION DOCUMENT

ECE 593: Fundamentals of Pre-Silicon Validation
Maseeh College of Engineering and Computer Science
Spring 2026



PROJECT TITLE:

DESIGN AND VERIFICATION OF
PARAMETERIZED SYSTOLIC ARRAY
ACCELERATOR

Project Members:

Hanisha Dhananjaya Produtur (hanisha@pdx.edu)

Nikhil Swarna (nikhilsw@pdx.edu)

Venkat Sai Sumanth koyada (koyada@pdx.edu)

Date: 4/15/2026

Github Link:

https://github.com/hanisha369/Parameterized_Systolic_Array_Accelerator

Design Features

1. Fully parameterized $N \times N$ systolic array. Matrix size, input data width, and accumulator width are all set by parameters N , $DATA_WIDTH$, and $ACCUM_WIDTH$.
2. Separate $DATA_WIDTH$ (default 8-bit) for inputs and $ACCUM_WIDTH$ (default 32-bit) for the result to prevent overflow during accumulation.
3. Input skew module staggers each input lane by 'i' clock cycles so that each matrix element arrives at the correct PE at the correct time.
4. Each Processing Element (PE) computes $accum += a_in \times b_in$ every cycle that $valid_in$ is high. Data passes horizontally (a) and vertically (b) to neighboring PEs
5. Valid signal is propagated diagonally through the PE grid so each PE only accumulates when its data is actually valid.
6. 4-state controller FSM: IDLE $\rightarrow$ LOAD $\rightarrow$ DRAIN $\rightarrow$ DONE. Counts N input cycles, waits $2N$ drain cycles, then pulses $result_valid$ for one cycle.
7. A clear signal is pulsed at the start of each new computation to zero all PE accumulators, allowing back-to-back matrix multiplications.
8. Result is output as a flat packed bus. Element $C[i][j]$ is at $result[(i*N+j)*ACCUM_WIDTH +: ACCUM_WIDTH]$.

Project Description

The **Parameterized Systolic Matrix Multiplication Accelerator** is a scalable hardware design implemented in SystemVerilog to efficiently perform matrix multiplication using a parallel systolic array architecture. Matrix multiplication is a fundamental and compute-intensive operation used in applications such as machine learning, signal processing, and scientific computing, where high performance is essential.

The design utilizes an $N \times N$ grid of Processing Elements (PEs), where each PE performs a multiply-accumulate (MAC) operation. Data flows through the array in a structured manner: elements of matrix A propagate horizontally, while elements of matrix B propagate vertically. As data moves through the array, each PE computes partial products and accumulates results over multiple clock cycles, enabling high parallelism and throughput.

To ensure correct timing and alignment of inputs, an input skewing mechanism is implemented using delay registers. A controller based on a finite state machine (FSM) manages different phases of operation, including data loading, computation, and pipeline draining, while generating control signals such as valid, ready, and result_valid.

The accelerator is fully parameterized, allowing configuration of array size and data precision, making it adaptable to various performance requirements. A UVM-based verification environment is used to validate functionality through constrained random testing and comparison with a reference model. Overall, the design demonstrates an efficient and scalable approach to hardware-accelerated matrix multiplication.

Matrix multiplication:

$$C[i][j] = \sum_k A[i][k] \times B[k][j]$$

Important Signals

Control Signals

No	Signal Name	Width	Purpose	Description
1	clk	1	Clock	The main clock signal used to synchronize all sequential operations in the accelerator.
2	rst_n	1	Active-Low Reset	Synchronous reset signal that initializes all modules to a known state.
3	start	1	Start Control	Initiates a new matrix multiplication operation.
4	clear	1	Clear Accumulator	Clears accumulation registers in all processing elements.
5	valid_in_upstream	1	Upstream Valid	Indicates valid input data from upstream source.
6	valid_in	1	Internal Valid	Internal signal to indicate valid data for processing.
7	valid_out	1	Output Valid	Indicates valid data has propagated through the array.
8	result_valid	1	Result Valid	Indicates final output result is ready.
9	ready	1	Ready	Indicates accelerator is ready to accept inputs.

Input Signals

No	Signal Name	Width	Purpose	Description
1	a_in	$N \times \text{DATA_WIDTH}$	Matrix Input A	Packed input bus carrying matrix A elements.
2	b_in	$N \times \text{DATA_WIDTH}$	Matrix Input B	Packed input bus carrying matrix B elements.
3	a_row	$N \times \text{DATA_WIDTH}$	Skewed Input A	Input to systolic array after skewing.
4	b_col	$N \times \text{DATA_WIDTH}$	Skewed Input B	Input to systolic array after skewing.

Output Signals

No	Signal Name	Width	Purpose	Description
1	result	$N \times N \times \text{ACCUM_WIDTH}$	Result Matrix	Final matrix multiplication output.
2	a_skewed	$N \times \text{DATA_WIDTH}$	SkewedA Output	Output from skew logic for matrix A.
3	b_skewed	$N \times \text{DATA_WIDTH}$	SkewedB Output	Output from skew logic for matrix B.
4	accum	ACCUM_WIDTH	Accumulated Result	Final accumulated output from PEs.

Design Signals

No	Signal Name	Width	Purpose	Description
1	ctrl_valid_in	1	Controller Valid	Valid signal generated by controller.
2	skew_valid_out	1	Skew Valid	Indicates skewed data is ready for array.
3	ctrl_clear	1	Controller Clear	Internal clear signal for PEs.

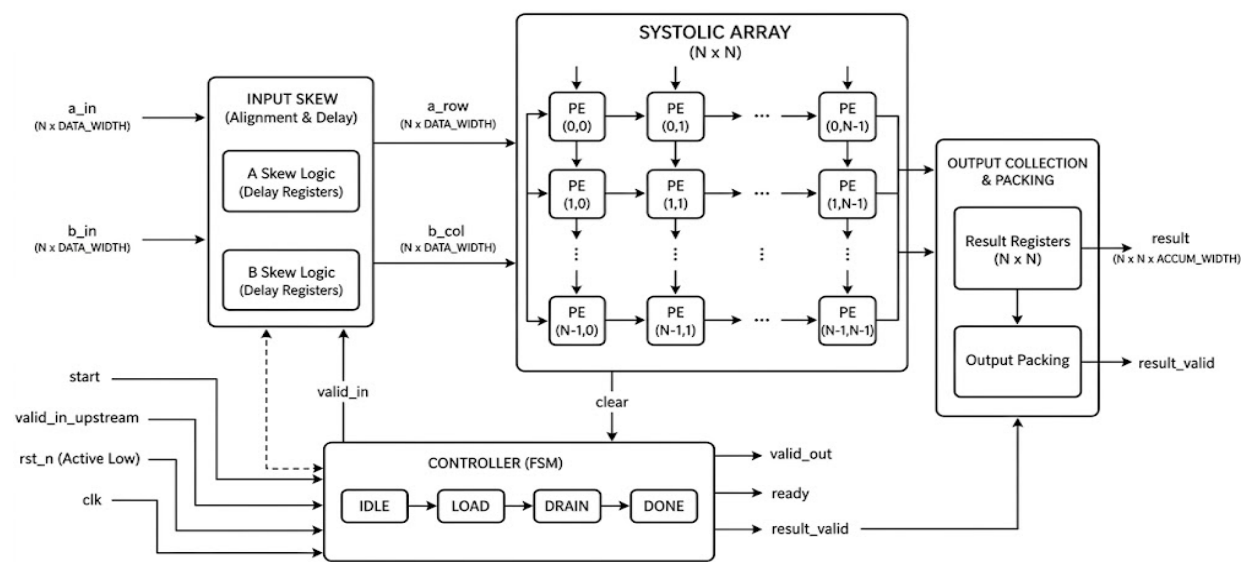
Systolic Array Signals

No	Signal Name	Width	Purpose	Description
1	a_wire	$N \times (N+1) \times \text{DATA_WIDTH}$	A Data Flow	Horizontal propagation of A data across array.
2	b_wire	$(N+1) \times N \times \text{DATA_WIDTH}$	B Data Flow	Vertical propagation of B data across array.
3	result_wire	$N \times N \times \text{ACCUM_WIDTH}$	PE Output	Stores outputs from each PE.
4	valid_diag	$N \times N$	Valid Pipeline	Propagates valid signal across array diagonally.
5	a_boundary	$N \times \text{DATA_WIDTH}$	A Boundary Input	Input boundary for A into array.
6	b_boundary	$N \times \text{DATA_WIDTH}$	B Boundary Input	Input boundary for B into array.

Processing Element Signals

No	Signal Name	Width	Purpose	Description
1	a_in	DATA_WIDTH	A Input	Input operand from left.
2	b_in	DATA_WIDTH	B Input	Input operand from top.
3	a_out	DATA_WIDTH	A Output	Forwarded A to next PE.
4	b_out	DATA_WIDTH	B Output	Forwarded B to next PE.
5	accum	ACCUM_WIDTH	Accumulator	Stores MAC result.
6	valid_in	1	Input Valid	Indicates valid input to PE.
7	valid_out	1	Output Valid	Propagates valid signal.
8	clear	1	Clear	Resets accumulator in PE.

Block Diagram:



References/Citations:

1. https://link.springer.com/chapter/10.1007/978-3-030-05677-3_16
2. <https://github.com/Devil-SX/Systolic-Array-for-Matrix-Multiplication>
3. https://www.youtube.com/watch?v=QvT_9vznrpg
4. <https://www.sciencedirect.com/science/article/pii/S1319157824002830>
5. <https://github.com/vonfel/fpga-systolic-accelerator>